

Звіт з лабораторної роботи № 7

Реалізація проекту з використанням патернів проектування

11 варіант

1. **Мета:** Розробити ієрархію класів згідно отриманого завдання описати їх алгоритмічною мовою C#. Навчитись використовувати патерни проектування.
2. **Завдання:**
 - 2.1. Траса трофі-рейду представляється у вигляді послідовності контрольних пунктів КП. Бувають два види КП: обов'язкового та необов'язкового взяття. Про кожне КП зберігається така інформація: точкою в діапазоні $-90.0^{\circ} \dots +90^{\circ}$ та довгота (число з плаваючою точкою в діапазоні $-180^{\circ} \dots +180^{\circ}$). Для КП з необов'язковим взяттям також зберігається значення штрафу за пропуск цього КП (число з плаваючою точкою, що становить час у годинах). Використовуючи підхід, застосовуваний у патерні Builder, розробіть фрагмент системи, забезпечує обробку списку КП. Реалізуйте ConcreteBuilder для: 1. Виведення списку КП у текстовому вигляді. Для кожного КП мають виводитися: порядковий номер; ім'я; координати; час штрафу або рядок «не зараховує СУ» для обов'язкових КП. 2. Підрахунки сумарного штрафу за всіма необов'язковими КП. 3. Виводу списку КП у віджеті з таблицею (наприклад, QTableView або аналогічному).
 - 2.2. Завдання 2. Використовуючи патерн Singleton, розробіть систему протоколювання подій в системі. Система повинна: - підтримувати 3 рівні важливості подій (нормальний, зауваження, помилка); - забезпечити фіксацію події (з подією фіксуються час, важливість, текстове повідомлення); - виводити на друк 10 останніх подій. Для реалізації необхідно реалізувати абстракцію (подію) та забезпечити її реалізацій. Дозволити генерування списку подій у довільній кількості та за вимогою користувача. Меню виклику подій представити класом.
 - 2.3. Завдання 3. Використовуючи патерн Абстрактний метод реалізувати проект для переміщення Людиною по різних визначних місцях. Передбачити різні типи місць та розраховувати вартість та час потрібний на відвідування в залежності від типу людини (дитина, дорослий, пенсіонер, студент). Потрібно передбачити наявність екскурсії (окрема абстаркція), яка включається в процес відвідування місця та має свою власну вартість. Дозволити користувачу скласти маршрут місць та отримати сумарну вартість та час затрачений на їхнє відвідування.

3. Виконання завдання

3.1. Виконання першого завдання:

3.1.1. Прописуємо інтерфейс «IBuilder». Ось його код:

```
interface IBuilder
{
    void AddCP(string Name, double width, double length, string type, double time);
}
```

3.1.2. Пояснення: У інтерфейсі існує метод AddCP, який дозволяє додати точку.

					ЛР3.14.KI-202.11.07		
Змін.	Аркуш	№ докум.	Підпис	Дата			
Розробив	Куліков М. М.				Абстрактні класи		Літ
Перевірив	Ковальчук С. О.						Аркуш
							Аркушів
Н. Контр.					ХПФК		
Затвер.							

3.1.3. Прописуємо клас «Checkpoint». Ось його код:

```
public class Checkpoint
{
    public string Name { get; set; }
    public double width { get; set; }
    public double length { get; set; }
    public string type { get; set; }
    public double timeOfPen { get; set; }

    public Checkpoint()
    {
        Name = "U Turn, Woods";
        width = 0;
        length = 0;
        type = "Required";
        timeOfPen = 0;
    }

    public Checkpoint(string Name, double width, double length, string type, double time)
    {
        this.Name = Name;
        this.width = width;
        this.length = length;
        this.type = type;
        this.timeOfPen = time;
    }
}
```

3.1.4. Пояснення: У класі є: атрибути: Name, width, length, type, timeofPen; конструктор Checkpoint(), Checkpoint(string Name...).

3.1.5. Прописуємо клас Road. Ось його код:

```
public class Road
{
    public List<Checkpoint> checkpoints = new List<Checkpoint>();

    public Road()
    {
    }

    public void AddCP(string name, double width, double length, string type, double time)
    {
        checkpoints.Add(new Checkpoint(name, width, length, type, time));
    }

    public string GetNameCP(int i)
    {
        return checkpoints[i].Name;
    }

    public double GetWidthCP(int i)
    {
        return checkpoints[i].width;
    }

    public double GetLengthCP(int i)
    {
        return checkpoints[i].length;
    }

    public double GetPenaltyCP(int i)
    {
        return checkpoints[i].timeOfPen;
    }

    public string GetTypeCP(int i)
    {
        return checkpoints[i].type;
    }
}
```

3.1.6. Пояснення: У класі є: атрибути: Лист з точками checkpoints; Конструктор Road(); Методи AddCP(...), GetNameCP(...), GetWidthCP(...), GetLength(...), GetPenalty(...), GetTypeCP(...).

3.1.7. Прописуємо клас ConcreteBuileder. Ось його код:

```
public class ConcreteBuilder: IBuilder
{
    public Road r1 = new Road();

    public ConcreteBuilder()
    {
        this.Reset();
    }

    public void Reset()
    {
        this.r1 = new Road();
    }

    public void AddCP(string Name, double width, double length, string type, double time)
    {
        if (type == "Required") time = 0;
        r1.AddCP(Name, width, length, type, time);
    }

    public double GetAllPen()
    {
        double allpen = 0;
        foreach (Checkpoint p in r1.checkpoints)
        {
            allpen += p.timeOfPen;
        }
        return allpen;
    }

    public void GetAllIntoTable(DataGridView d1)
    {
        int i = 0;

        if (r1.checkpoints.Count == 0) d1.RowCount = 1;
        else d1.RowCount = r1.checkpoints.Count + 1;

        foreach (Checkpoint p in r1.checkpoints)
        {
            d1.Rows[i].Cells[0].Value = i + 1;
            d1.Rows[i].Cells[1].Value = r1.GetNameCP(i);
            d1.Rows[i].Cells[2].Value = r1.GetWidthCP(i);
            d1.Rows[i].Cells[3].Value = r1.GetLengthCP(i);
            d1.Rows[i].Cells[4].Value = r1.GetPenaltyCP(i);
            d1.Rows[i + 1].Cells[4].Value = GetAllPen();
            d1.Rows[i].Cells[5].Value = r1.GetTypeCP(i);
            i++;
        }
    }

    public void GetAllText(ListBox l1)
    {
        int i = 0;
        foreach (Checkpoint p in r1.checkpoints)
        {
            l1.Items.Add(i + 1 + " | " + r1.GetNameCP(i) + " | " + r1.GetWidthCP(i) + " " + r1.GetLengthCP(i) +
                " | " + r1.GetPenaltyCP(i));
            i++;
        }
    }

    public Road GetRoad()
    {
        Road result = this.r1;
        Reset();
        return result;
    }
}
```

3.1.8. Пояснення: У класі є: атрибут Road; конструктор ConcreteBuilder(); методи Reset(), AddCP(...), GetAllCP(...), GetAllPen(...), GetAllIntoTable(...), GetAllTextList(...), GetRoad().

3.1.9. Створимо дві форми. У першій ми можемо додавати нові точки, у другій ж виводимо усі точки (текстом). Ось вигляд першої форми:

Position	Name	Width	Length	Penalty	Is Required?
1	Woods_	10	100	0	Required
2	Woods U-turn	20	100	0.001	Not Required
3	Woods Finish	50	140	0	Required

3.1.10. Ось вигляд другої форми:

```
1 | Woods | 10 | 100 | 0
2 | Woods U-turn | 20 | 100 | 0.001
3 | Woods Finish | 50 | 140 | 0
```

3.1.11. Код першої форми:

```
public partial class Form1 : Form
{
    dynamic builder = new ConcreteBuilder();
    public Form1()
    {
        InitializeComponent();
    }

    private void Form1_Load(object sender, EventArgs e)
    {
        builder.AddCP("Woods_", 10, 100, "Required", 0);
        builder.AddCP("Woods U-turn", 20, 100, "Not Required", 0.001);
        builder.AddCP("Woods Finish", 50, 140, "Required", 0);

        builder.GetAllIntoTable(dataGridView1);
    }

    private void yciHaEкранToolStripMenuItem_Click(object sender, EventArgs e)
    {
        Form2 f2 = new Form2(builder);
        f2.ShowDialog();
    }

    private void dataGridView1_CellEndEdit(object sender, DataGridViewCellEventArgs e)
    {
        int i = dataGridView1.CurrentRow.Index;
        if ((dataGridView1.Rows[i].Cells[1].Value != null) &&
            (dataGridView1.Rows[i].Cells[2].Value != null) && (dataGridView1.Rows[i].Cells[3].Value != null) &&
            (dataGridView1.Rows[i].Cells[4].Value != null) && (dataGridView1.Rows[i].Cells[5].Value != null))
        {
            builder.AddCP(dataGridView1.Rows[i].Cells[1].Value.ToString(),
                Convert.ToDouble(dataGridView1.Rows[i].Cells[2].Value),
                Convert.ToDouble(dataGridView1.Rows[i].Cells[3].Value),
                dataGridView1.Rows[i].Cells[5].Value.ToString(),
                Convert.ToDouble(dataGridView1.Rows[i].Cells[4].Value));
            builder.GetAllIntoTable(dataGridView1);
        }
    }
}
```

3.1.12. Код другої форми:

```

public partial class Form2 : Form
{
    dynamic builder = new ConcreteBuilder();
    public Form2(dynamic b)
    {
        InitializeComponent();
        builder = b;
    }

    private void Form2_Load(object sender, EventArgs e)
    {
        builder.GetAllText(listBox1);
    }
}

```

3.2. Виконання другого завдання:

3.2.1. Прописуємо клас Singleton. Ось його код:

```

public class Singleton
{
    private static Singleton occasion;
    private Singleton()
    {
    }

    public static Singleton GetInstance()
    {
        if (occasion == null)
        {
            occasion = new Singleton();
        }
        return occasion;
    }

    public string Protocol(Event ev)
    {
        return ev.Name + " | " + ev.Importance + " | " + ev.data;
    }
}

```

3.2.2. Пояснення: Клас має: атрибут occasion, конструктор Singleton(), методи GetInstance(), Protocol(...).

3.2.3. Прописуємо клас Event. Ось його код:

```

public class Event
{
    Random rnd = new Random();
    public string Name { get; set; }
    public string Importance { get; set; } = "Виконано";
    public TimeSpan data { get; set; }

    public Event()
    {
    }

    public Event(TimeSpan Time)
    {
        this.data = Time;
    }

    public void SetName()
    {
        int d = rnd.Next(0, 20);

        if (d % 2 == 1)
        {
            Name = "Відкриття файлу";
        }
        else if (d % 2 == 0)
        {
            Name = "Збереження файлу";
        }
        else
        {
            Name = "Видалення файлу";
        }
    }
}

```

3.2.4. Пояснення: клас має: атрибути: Name, Importance, data, конструктори Event(), Event(...), метод SetName().

3.2.5. Створимо форму, куди будемо виводити події і їх стан. Ось її вигляд:

					ЛР3.14.KI-202.11.07	Арк.
Вим	Арк.	№ докум.	Підпис	Дата		5

Важливість	Час
Відкриття файлу	20:51:17
Відкриття файлу	20:51:17
Відкриття файлу	20:51:17
Відкриття файлу	20:51:17
Відкриття файлу	20:51:17
Відкриття файлу	20:51:17
Відкриття файлу	20:51:23
Збереження файлу	20:51:24

Кількість: 1

Генерувати

Запустити

Відкриття файлу | Виконано | 20:51:17
Збереження файлу | Виконано | 20:51:24

3.2.6. Код цієї форми:

```
public partial class Form1 : Form
{
    public List<Event> events = new List<Event>();
    public Form1()
    {
        InitializeComponent();
    }

    public void CreateEV(int i)
    {
        int count = 0;
        if (events.Count <= 0) count = 0;
        else count = events.Count;
        for (int j = count; j < count+i; j++)
        {
            TimeSpan Time = new TimeSpan(DateTime.Now.Hour, DateTime.Now.Minute, DateTime.Now.Second);
            events.Add(new Event(Time));
            events[j].SetName();

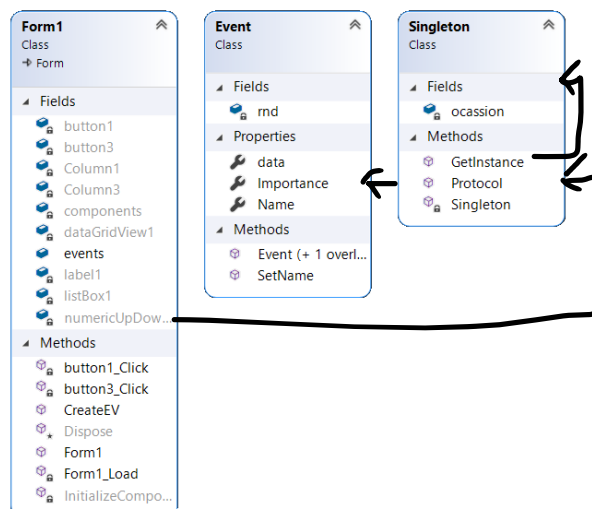
            dataGridView1.RowCount++;
            dataGridView1.Rows[dataGridView1.RowCount - 2].Cells[0].Value = events[j].Name;
            dataGridView1.Rows[dataGridView1.RowCount - 2].Cells[1].Value = events[j].data;
        }
    }

    private void button1_Click(object sender, EventArgs e)
    {
        int num = Convert.ToInt32(numericUpDown1.Value);
        CreateEV(num);
    }

    private void Form1_Load(object sender, EventArgs e)
    {
    }

    private void button3_Click(object sender, EventArgs e)
    {
        int ind = dataGridView1.CurrentRow.Index;
        listBox1.Items.Add(Singleton.GetInstance().Protocol(events[ind]));
    }
}
```

3.2.7. Діаграма залежності класів:



3.3. Виконання третього завдання:

3.3.1. Прописуємо інтерфейс IPlace. Ось його код:

```
public interface IPlace
{
    double getPrice();
    double getTime();
    string getName();
    string getPerson();
}
```

3.3.2. Пояснення: у інтерфесі прописані методи GetPrice(), GetTime(), GetName(),GetPerson().

3.3.3. Прописуємо інтерфейс IAbstractFactory. Ось його код:

```
public interface IAbstractFactory
{
    IPlace CreateMuseum(string Person);
    IPlace CreatePark(string Person);
    IPlace CreateMonument(string Person);
    Excursion CreateExcursionA(string Person);
}
```

3.3.4. Пояснення: у інтерфейсі прописані методи CreateMuseum(...), CreatePark(...), CreateMonument(...), CreateExcursion(...).

3.3.5. Прописуємо клас Monument. Ось його код:

```
public class Monument: IPlace
{
    public string Name { get; set; }
    public double time { get; set; }
    public string person { get; set; }
    public double price { get; set; }
    public Monument() { }
    public Monument(string Name, double time, string Person)
    {
        this.Name = Name;
        this.time = time;
        this.person = Person;
        setPrice(Person);
    }

    public void setPerson(string Person) { this.person = Person; }
    public void setPrice(string Person)
    {
        if (Person.Equals("Дитина"))
        {
            price = 10;
        }
        else if (Person.Equals("Студент"))
        {
            price = 15;
        }
        else if (Person.Equals("Дорослий"))
        {
            price = 20;
        }
        else if (Person.Equals("Пенсіонер"))
        {
            price = 16;
        }
    }
    public double getPrice() { return price; }
    public double getTime() { return 0.3; }
    public string getName() { return Name; }
    public string getPerson() { return person; }
}
```

3.3.6. Пояснення: у класі описуються методи, які вказані в інтерфейсі IPlace. Окрім цього, клас має: атрибути Name, time, person, price, конструктори Monument(), Monument(...), методи SetPerson(...), setPrice(...).

3.3.7. Прописуємо клас Museum. Ось його код:

```

public class Museum: IPlace
{
    public string Name { get; set; }
    public double time { get; set; }
    public string person { get; set; }
    public double price { get; set; }
    public Museum() { }
    public Museum(string Name, double time, string Person)
    {
        this.Name = Name;
        this.time = time;
        this.person = Person;
        setPrice(Person);
    }

    public void setPerson(string Person) { this.person = Person; }
    public void setPrice(string Person)
    {
        if (Person.Equals("Дитина"))
        {
            price = 10;
        }
        else if (Person.Equals("Студент"))
        {
            price = 15;
        }
        else if (Person.Equals("Дорослий"))
        {
            price = 20;
        }
        else if (Person.Equals("Пенсіонер"))
        {
            price = 16;
        }
    }
    public double getPrice()
    {
        return price;
    }
    public double getTime()
    {
        return 1;
    }
    public string getName()
    {
        return Name;
    }
    public string getPerson()
    {
        return person;
    }
}

```

3.3.8. Пояснення: у класі описуються методи, які вказані в інтерфейсі IPlace. Окрім цього, клас має: атрибути Name, time, person, price, конструктори Monument(), Monument(...), методи SetPerson(...), setPrice(...).

3.3.9. Прописуємо клас Park. Ось його код:


```

public class Park: IPlace
{
    public string Name { get; set; }
    public double time { get; set; }
    public string person { get; set; }
    public double price { get; set; }

    public Park() { }

    public Park(string Name, double time, string Person)
    {
        this.Name = Name;
        this.time = time;
        this.person = Person;
        setPrice(Person);
    }

    public void setPerson(string Person) { this.person = Person;}
    public void setPrice(string Person)
    {
        if (Person.Equals("Дитина"))
        {
            price = 10;
        }
        else if (Person.Equals("Студент"))
        {
            price = 15;
        }
        else if (Person.Equals("Дорослий"))
        {
            price = 20;
        }
        else if (Person.Equals("Пенсіонер"))
        {
            price = 0;
        }
        price = 0;
    }
    public double getPrice()
    {
        return price;
    }
    public double getTime()
    {
        return 0.5;
    }
    public string getName()
    {
        return Name;
    }
    public string getPerson()
    {
        return person;
    }
}

```

3.3.10. Пояснення: у класі описуються методи, які вказані в інтерфейсі IPlace. Окрім цього, клас має: атрибути Name, time, person, price, конструктори Monument(), Monument(...), методи SetPerson(...), setPrice(...).

3.3.11. Прописуємо клас Excursion. Ось його код:

```

public class Excursion
{
    public double price { get; set; }

    public void setPrice(double byCity)
    {
        price += museum.getPrice() + park.getPrice() + monument.getPrice() + byCity;
    }

    public Museum museum { get; set; }
    public Park park { get; set; }
    public Monument monument { get; set; }
}

```

3.3.12. Пояснення: Клас має: атрибут price, museum, park, monument, метод setPrice(...).

3.3.13. Прописуємо клас Khmelnytskyi, який буде виступати як одна з абстрактних фабрик. Ось його код:

```

public class Khmelnytskiy: IAbstractFactory
{
    public List<IPlace> places = new List<IPlace>();
    public IPlace CreateMonument(string Person)
    {
        return new Museum("Пам'ятник Івану Франку", 0, Person);
    }

    public IPlace CreatePark(string Person)
    {
        return new Park("Полісся", 0, Person);
    }

    public IPlace CreateMuseum(string Person)
    {
        return new Museum("Хмельницький музей історії фотографії", 1, Person);
    }

    public Excursion CreateExcursionA(string Person)
    {
        Excursion ex = new Excursion();
        ex.monument = new Monument("Пам'ятник Богдану Хмельницькому", 1, Person);
        ex.park = new Park("ArtPlace", 1, Person);
        ex.museum = new Museum("Музей історії міста", 2, Person);
        ex.monument.setPrice(Person);
        ex.park.setPrice(Person);
        ex.museum.setPrice(Person);
        ex.setPrice(50);
        return ex;
    }
}

```

3.3.14. Пояснення: У класі реалізуються методи з IAbstractFactory, окрім цього, клас має атрибут places. Аналогічно і з класами, описаними нижче.

3.3.15. Аналогічно прописуємо класи Lviv і Kyiv. Ось код класу Lviv:

```

public class Lviv: IAbstractFactory
{
    public List<IPlace> places = new List<IPlace>();
    public IPlace CreateMonument(string Person)
    {
        return new Museum("Львівська писанка", 1, Person);
    }

    public IPlace CreatePark(string Person)
    {
        return new Park("Лісовий парк", 0.5, Person);
    }

    public IPlace CreateMuseum(string Person) { return new Museum("Львівський музей історії", 1, Person); }

    public Excursion CreateExcursionA(string Person)
    {
        Excursion ex = new Excursion();
        ex.monument = new Monument("Львівський архітектор", 1, Person);
        ex.park = new Park("Парк культури", 1, Person);
        ex.museum = new Museum("Архітектурна галерея мистецтв", 2, Person);
        ex.monument.setPrice(Person);
        ex.park.setPrice(Person);
        ex.museum.setPrice(Person);
        ex.setPrice(250);
        return ex;
    }
}

```

3.3.16. Ось код класу Kyiv:

```

public class Kyiv: IAbstractFactory
{
    public IPlace CreateMonument(string person)
    {
        return new Museum("Майдан Незалежності", 0, person);
    }

    public IPlace CreatePark(string Person) { return new Park("Парк ім. Гетьмана", 0, Person); }
    public IPlace CreateMuseum(string Person) { return new Museum("Музей української історії", 1, Person); }
    public Excursion CreateExcursionA(string Person)
    {
        Excursion ex = new Excursion();
        ex.monument = new Monument("Пам'ятник Перемоги", 1, Person);
        ex.park = new Park("Парк сім'ї Коцюбинських", 1, Person);
        ex.museum = new Museum("Музей української історії", 2, Person);
        ex.monument.setPrice(Person);
        ex.park.setPrice(Person);
        ex.museum.setPrice(Person);
        ex.setPrice(200);
        return ex;
    }
}

```

3.3.17. Створимо форму, куди будуть виводитись наші дані. Ось її вигляд:

3.3.18. Ось її код:

```
public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();
        comboBox1.SelectedIndex = 0;
        comboBox2.SelectedIndex = 0;
        comboBox3.SelectedIndex = 0;
    }

    IAbstractFactory factory = null;
    IPlace place = null;

    private void button1_Click(object sender, EventArgs e)
    {
        if (((string)comboBox1.SelectedItem).Equals("Музей")) place =
            factory.CreateMuseum(((string)comboBox2.SelectedItem));
        else if (((string)comboBox1.SelectedItem).Equals("Парк")) place =
            factory.CreatePark(((string)comboBox2.SelectedItem));
        else place = factory.CreateMonument(((string)comboBox2.SelectedItem));

        dataGridView1.RowCount++;
        dataGridView1.Rows[dataGridView1.RowCount - 2].Cells[0].Value = place.getPerson();
        dataGridView1.Rows[dataGridView1.RowCount - 2].Cells[1].Value = place.getName();
        dataGridView1.Rows[dataGridView1.RowCount - 2].Cells[2].Value = place.getPrice();
        dataGridView1.Rows[dataGridView1.RowCount - 2].Cells[3].Value = place.getTime();

        UPDGr();
    }

    private void UPDGr()
    {
        double sum = 0, sumt = 0;

        for (int j = 0; j < dataGridView1.RowCount; j++)
        {
            sum += Convert.ToDouble(dataGridView1.Rows[j].Cells[2].Value);
            sumt += Convert.ToDouble(dataGridView1.Rows[j].Cells[3].Value);
        }

        textBox1.Text = sum.ToString();
        textBox2.Text = sumt.ToString();
    }

    private void button2_Click(object sender, EventArgs e)
    {
        Excursion ex = null;
        ex = factory.CreateExcursionA(((string)comboBox2.SelectedItem));

        dataGridView1.RowCount++;
        dataGridView1.Rows[dataGridView1.RowCount - 2].Cells[0].Value = ex.museum.getPerson();
        dataGridView1.Rows[dataGridView1.RowCount - 2].Cells[1].Value = ex.museum.getName();
        dataGridView1.Rows[dataGridView1.RowCount - 2].Cells[2].Value = ex.museum.getPrice();
        dataGridView1.Rows[dataGridView1.RowCount - 2].Cells[3].Value = ex.museum.getTime();
        dataGridView1.RowCount++;
        dataGridView1.Rows[dataGridView1.RowCount - 2].Cells[0].Value = ex.park.getPerson();
        dataGridView1.Rows[dataGridView1.RowCount - 2].Cells[1].Value = ex.park.getName();
        dataGridView1.Rows[dataGridView1.RowCount - 2].Cells[2].Value = ex.park.getPrice();
        dataGridView1.Rows[dataGridView1.RowCount - 2].Cells[3].Value = ex.park.getTime();
        dataGridView1.RowCount++;
        dataGridView1.Rows[dataGridView1.RowCount - 2].Cells[0].Value = ex.monument.getPerson();
        dataGridView1.Rows[dataGridView1.RowCount - 2].Cells[1].Value = ex.monument.getName();
        dataGridView1.Rows[dataGridView1.RowCount - 2].Cells[2].Value = ex.monument.getPrice();
        dataGridView1.Rows[dataGridView1.RowCount - 2].Cells[3].Value = ex.monument.getTime();

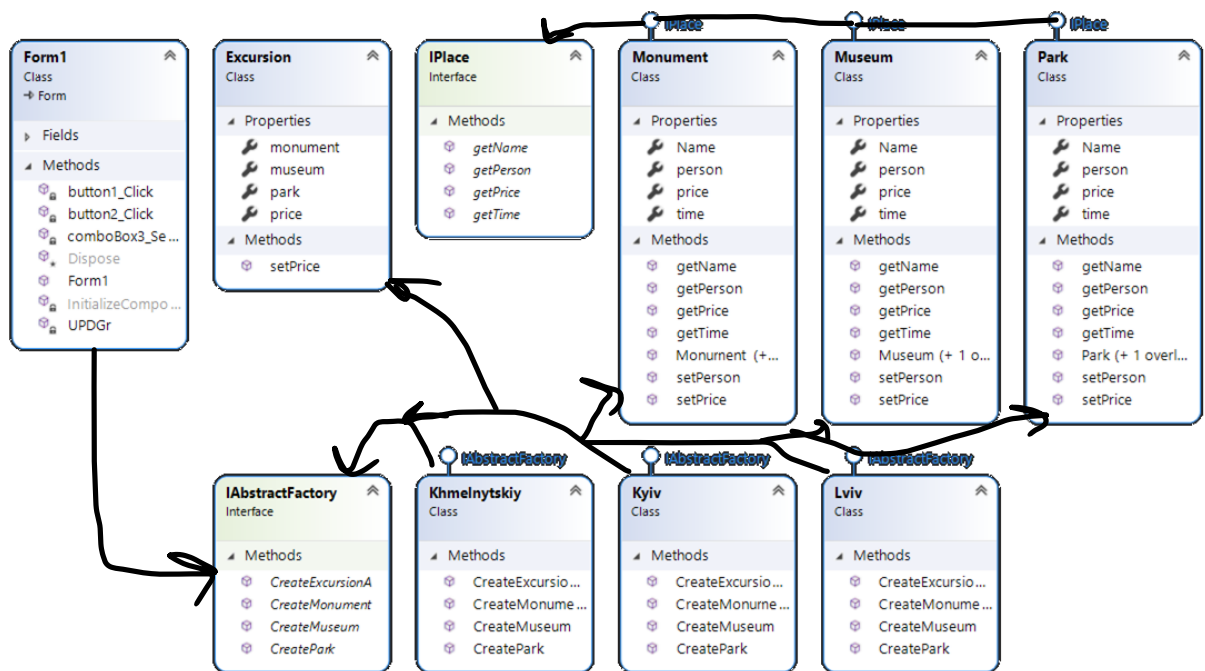
        UPDGr();
    }
}
```

```

private void comboBox3_SelectedIndexChanged(object sender, EventArgs e)
{
    switch (((string)comboBox3.SelectedItem))
    {
        case "Київ":
        {
            factory = new Kyiv();
            break;
        }
        case "Львів":
        {
            factory = new Lviv();
            break;
        }
        case "Хмельницький":
        {
            factory = new Khmelnytskyi();
            break;
        }
    }
}
}

```

3.3.19. Ось діаграма залежності класів:



4. Висновок: Ми розробили ієрархію класів згідно отриманого завдання, описали їх алгоритмічною мовою C# та навчились використовувати патерни проектування.